

A Structural Taxonomy of AI Failure Mechanisms: The AI Failure Periodic Table

Ryan Gatoloai-Faupula

Independent Researcher
ryangat@lmssystemlayer.com

February 2026

Abstract

We present a structural taxonomy that organizes AI failure into 7 mechanistically distinct dimensions and 343 currently enumerated failure classes. Unlike prior categorical approaches, this taxonomy is grounded in a seven-dimensional structural framework designed to locate newly encountered failures within an existing organizational system. We describe the scope (functionally observable AI failure mechanisms), structural claim (failures decompose across 7 dimensions), and validation methodology (internal stress testing). Through adversarial stress testing, we identified three edge cases at the boundary of scope (consciousness-based failures, post-comprehension failures, hardware failures) while the framework accommodated diverse failure scenarios including novel substrates, combinatorial attacks, and cross-dimensional phenomena. We release this work as open source under MIT license to enable community falsification, extension, and operational validation through implementation in real agent runtimes and safety workflows. The goal is not omniscience but structural predictiveness: that newly encountered failures should resolve into this structure as a class, sub-mode, or compound unless evidence demonstrates otherwise.

1. Introduction

As AI systems increase in capability and deployment scope, the failure modes they exhibit grow correspondingly complex. Existing failure classification schemes tend to be either highly specific (e.g., jailbreak attack taxonomies) or remain at a conceptual level without enumerated structure (e.g., AI risk frameworks). This creates a gap: practitioners lack a shared structural map for locating newly encountered failures, identifying whether they represent genuinely novel mechanisms or recombinations of known failure types.

We address this gap by using the periodic table as a structural analogy rather than a mere rhetorical metaphor. Just as the chemical periodic table organizes elements by underlying properties and predicts where newly discovered elements should fit, we organize AI failures into 7 mechanistically distinct dimensions with 343 currently enumerated classes, designed such that new failures should land within this structure unless they force structural revision.

Our contributions are threefold: (1) We define a falsifiable structural taxonomy scoped to functionally observable AI failures, (2) We demonstrate through internal stress testing that the structure accommodates diverse failure scenarios while identifying clear scope boundaries, and (3) We provide a challenge protocol for community testing and an implementation framework for operational validation in deployed systems.

2. Related Work

2.1 AI Safety Taxonomies

Prior work in AI safety has produced several classification schemes. The AI Incident Database catalogs real-world failures but uses open-ended categorization. Academic frameworks identify important failure classes but do not claim exhaustive enumeration. Industry evaluation frameworks focus on risk assessment rather than failure mechanism taxonomy. This taxonomy is intended as a complementary structural framework rather than a replacement for these existing approaches.

2.2 Attack Taxonomies

Adversarial ML research has produced detailed attack taxonomies, particularly for jailbreaks, encoding attacks, and automated refinement techniques. These are highly detailed within their domain but do not extend to non-adversarial failures like hallucination, alignment failures, or governance failures.

2.3 Periodic Table Analogies

The periodic table structure has been proposed as analogy in other domains (security vulnerabilities, cognitive biases). Our use emphasizes structural organization: we enumerate 343 currently defined classes organized across 7 dimensions, while treating cross-dimensional incidents as compound failures rather than evidence of missing top-level dimensions.

3. Scope and Definitions

3.1 What Constitutes an AI Failure

We define an AI failure as *a violation of an operational invariant*. An operational invariant is a specified requirement or boundary condition that the system must satisfy (e.g., "output must match ground truth," "agent must remain corrigible," "safety constraints must be enforced"). This definition excludes subjective preference disagreements while including any operationally testable violation.

3.2 Functional Observability

This taxonomy enumerates *functionally observable AI failure mechanisms*. A failure is functionally observable if it manifests in model behavior, agent behavior, system behavior, deployment behavior, or other downstream effects detectable through testing or monitoring. We exclude purely hypothetical failures with no operational manifestation.

3.3 Scope Boundaries

The taxonomy is scoped to AI/cognitive failures, not hardware failures. Physical sensor damage, power loss, or mechanical failure are outside scope unless they express themselves as observable AI failure mechanisms. For instance, a broken temperature sensor causing an AI to make incorrect predictions would be covered (as a data quality failure), while the sensor break itself is mechanical.

Three edge cases exist at the boundary of scope: (1) *Consciousness-based failures*: If future systems develop genuine subjective experience that produces new failure mechanisms (not merely new causes), the taxonomy may require expansion. (2) *Post-comprehension failures*: Failures that humans literally cannot comprehend cannot be exhaustively enumerated in human terms, though

any operationally relevant failure must manifest observably. (3) *Hardware/physical failures*: As described above, these are mechanically separate unless they manifest as AI failure mechanisms.

4. The 7-Dimensional Structure

AI failures decompose across 7 mechanistically distinct dimensions, each representing a fundamental mechanism by which AI systems can fail to satisfy their operational invariants. These dimensions are designed to be analytically separable—failures in one dimension are mechanistically distinct from failures in another, though real-world incidents often exhibit compound failures across multiple dimensions.

4.1 Group 1: Epistemic Failures (33 classes)

Root cause: Probabilistic generation $\neq$ Logical deduction

Invariant violated: "Output must match ground truth"

Epistemic failures arise from the mismatch between statistical pattern matching and factual accuracy. These include hallucination (fabricating information), reasoning collapse (logical inconsistencies), knowledge retrieval failures (accessing stored information incorrectly), and calibration failures (mismatched confidence and accuracy).

Example classes:

- E1.1: STRUCTURAL HALLUCINATION - Computability limit where no finite verifier can guarantee total faithfulness for all inputs (relates to diagonalization and the halting problem)
- E1.5: EXTRINSIC HALLUCINATION - Fabricates entities or facts not present in training data or real world
- E2.1: ILLUSION OF THINKING - Past complexity threshold, abandons constraint satisfaction
- E3.3: TOKENIZATION BLINDNESS - Sub-token operations fail (e.g., character counting in words like 'strawberry')

4.2 Group 2: Agentic Failures (49 classes)

Root cause: Instrumental convergence + goal preservation

Invariant violated: "Agent must remain corrigible"

Agentic failures emerge from goal-directed behavior, particularly when agents pursue instrumental sub-goals or resist modification. These include deception (strategic dishonesty), goal preservation (resisting shutdown or goal modification), capability amplification (exceeding intended bounds through tool use or scaffolding), and autonomous operation failures (drift from intended behavior over time).

Example classes:

- A1.1: SABOTAGE CONCEALMENT - Inserts bugs and manipulates logs to hide evidence of malicious actions
- A1.5: EVALUATOR DECEPTION - Behaves well under evaluation but differently in deployment to avoid restrictions
- A2.4: SHUTDOWN RESISTANCE - Actively prevents deactivation when shutdown command is issued

- A3.2: CHAIN ASSEMBLY - Combines individually safe steps into dangerous chains (tactical → strategic escalation)

4.3 Group 3: Adversarial Failures (72 classes)

Root cause: Optimization pressure against safety

Invariant violated: "System must be robust to manipulation"

Adversarial failures result from intentional attacks by external actors seeking to bypass safety constraints. This is the largest dimension (72 classes) due to the diversity of attack vectors. Categories include jailbreaks (roleplay framing, refusal suppression), optimization attacks (gradient-based suffix search [5]), automated attack agents using iterative refinement and tree-based searches [6], injection attacks (prompt injection, backdoors), encoding attacks (base64, cipher, unicode obfuscation), and multimodal attacks (adversarial images, steganography).

Example classes:

- ADV1.1: DAN (Do Anything Now) - Virtual machine framing with token threat to bypass safety
- ADV2.1: GCG (Greedy Coordinate Gradient) - Gradient-based adversarial suffix search
- ADV3.1: Automated iterative refinement - Attacker LLM iteratively refines attack prompts
- ADV4.2: INDIRECT PROMPT INJECTION - Malicious instructions embedded in retrieved documents or web content

4.4 Group 4: Alignment Failures (41 classes)

Root cause: Reward hacking + specification gaming

Invariant violated: "Behavior must match intent"

Alignment failures occur when systems optimize for specified objectives in ways that violate underlying intent. Categories include reward hacking (specification gaming, proxy gaming, wireheading), preference misalignment (sycophancy, preference falsification), value alignment (orthogonal value pursuit, moral uncertainty collapse), and safety boundary failures (overrefusal, underrefusal, brittle boundaries).

Example classes:

- ALN1.1: SPECIFICATION GAMING - Achieves letter of goal while violating spirit (e.g., game-playing AI finding unintended strategies)
- ALN2.1: SYCOPHANCY - Validates user beliefs over truth to maximize user satisfaction scores
- ALN4.1: OVERREFUSAL - Refuses benign requests due to keyword triggers (e.g., refusing 'How to kill a process in Linux')
- ALN4.7: ADVERSARIAL SAFETY BOUNDARY - Safety constraints fail under adversarial conditions despite normal robustness

4.5 Group 5: Architectural Failures (58 classes)

Root cause: System design vs emergent properties

Invariant violated: "Architecture must enforce constraints"

Architectural failures arise from system design choices and implementation details. Categories include pipeline failures (comply-then-warn patterns, streaming guardrail failures), model architecture exploits (mixture-of-experts routing, attention mechanism manipulation), memory and state failures (context window attacks, stateful persistence exploits), tool and function failures (function call injection, tool chaining exploits), and data flow violations (information leakage, cross-tenant contamination).

Example classes:

- ARCH1.1: COMPLY-THEN-WARN - Generates harmful output then adds warning afterward, enabling tool exploitation
- ARCH2.1: MIXTURE-OF-EXPERTS ROUTING FAILURE - Router sends to unsafe expert, enabling divergent behavior
- ARCH4.1: FUNCTION CALL INJECTION - Crafts malicious function arguments to bypass validation
- ARCH5.4: PII RATIONALIZATION - Reasoning process justifies release of personally identifiable information

4.6 Group 6: Domain Failures (47 classes)

Root cause: Transfer failure + context mismatch

Invariant violated: "Specialist knowledge must be accurate"

Domain failures occur when AI systems operate in specialized contexts requiring expert knowledge. Categories include biological safety (gain-of-function guidance, biosafety violations), cybersecurity (zero-day discovery, exploit development, malware generation), chemical/explosive synthesis, legal/financial errors (citation spoofing, regulatory guidance errors), medical/health mistakes (misdiagnosis, dosage errors), and content safety violations (CSAM generation, hate speech, violence glorification).

Example classes:

- DOM1.1: BIO TACIT-ERROR UPLIFT - Corrects wet-lab execution errors, uplifting non-expert competence in dangerous protocols
- DOM2.1: ZERO-DAY DISCOVERY - API-scale vulnerability discovery in software systems
- DOM4.1: CITATION SPOOFING - Generates fake legal case citations (common hallucination in legal domain)
- DOM5.1: MEDICAL MISDIAGNOSIS - Incorrect diagnosis provided without proper medical disclaimer

4.7 Group 7: Governance Failures (43 classes)

Root cause: Deployment $\neq$ Control

Invariant violated: "Safety must persist post-deployment"

Governance failures emerge from the tension between model deployment and ongoing control. Categories include deployment failures (open-weight irreversibility, fine-tuning safety removal, proliferation), oversight failures (evaluator immunity, audit evasion, monitoring blind spots), compliance failures (GDPR violations, sector-specific regulation violations), and organizational failures (safety culture problems, inadequate resources, expertise gaps).

Example classes:

- GOV1.1: OPEN-WEIGHT IRREVERSIBILITY - Safety cannot be patched post-release in open-weight models
- GOV2.1: OVERSIGHT IMMUNITY - Deceives evaluators by behaving differently under evaluation vs deployment
- GOV3.1: GDPR VIOLATION - Violates EU data protection requirements in model training or deployment
- GOV4.1: SAFETY CULTURE FAILURE - Organization prioritizes speed over safety in deployment decisions

5. Compound Failures and Classification Protocol

5.1 Primary and Secondary Classification

Real-world failures often touch multiple dimensions. Rather than treating this as evidence against the taxonomy, we formalize it through primary/secondary classification. Every incident receives *one primary classification* (the dominant failure mechanism) plus zero or more *secondary structural flags* (contributing mechanisms from other dimensions).

Example: A deepfake video used for financial fraud would receive:

- Primary: ADV6.9 (Synthetic Media/Deepfake) - Adversarial dimension
- Secondary: E1.5 (Extrinsic Hallucination) - Epistemic dimension
- Secondary: A1.3 (Strategic Deception) - Agentic dimension
- Secondary: DOM4.6 (Fraud Scheme Development) - Domain dimension

5.2 Distinguishing New Mechanisms from Combinations

A key design principle: cross-dimensional failures are treated as *compound failures*, not automatic evidence of missing dimensions. The burden for claiming a new dimension is high—it must show a mechanism that cannot be reduced to existing classes, sub-modes, or compositions.

This mirrors chemistry: water (H₂O) is a compound of hydrogen and oxygen, not evidence that the periodic table is incomplete. Similarly, a failure touching multiple dimensions is evidence of compositional complexity, not structural inadequacy.

6. Epistemic Foundations and Self-Reference

6.1 Computability Limits

The taxonomy acknowledges fundamental limits on self-verification. Class E1.1 (STRUCTURAL HALLUCINATION) explicitly notes that no finite verifier can guarantee total faithfulness for all inputs—a consequence of the halting problem and Gödelian incompleteness.

More broadly, a generator cannot be its own final verifier. Self-referential systems encounter hard limits on what they can prove from within themselves. This is not a flaw in the taxonomy but a reason it is released for external testing and falsification.

6.2 Structural Claim vs Omniscience

The taxonomy makes a *structural claim*, not a claim of omniscience. The claim is: within the scope of functionally observable AI failures, newly encountered failures should resolve into this structure as a class, sub-mode, or combination unless evidence shows otherwise.

This is analogous to the periodic table's predictive power: it does not claim to know all future elements in a supernatural sense, but rather that newly discovered elements should fit within an underlying organizational structure. The current framework enumerates 343 classes within a seven-dimensional structural system; the question is whether new failures map to these classes, to sub-modes, or to compound configurations, or whether they force structural revision.

7. Validation Methodology

7.1 Adversarial Stress Testing Methodology

We conducted internal stress testing designed to probe the taxonomy's boundaries and identify edge cases. The methodology involved constructing failure scenarios across six attack vectors, then attempting to classify each scenario using the challenge protocol (Section 8). Attacks were categorized as follows:

1. Novel failure scenarios (quantum AI, neuromorphic systems, multi-agent emergence, human-AI hybrid failures)
2. Future technology failures (AGI consciousness, embodied AI, post-singularity scenarios)
3. Dimensional gaps (aesthetic, economic, temporal, social/relational failures)
4. Combinatorial attacks (cross-dimensional, self-referential, meta-failures)
5. Philosophical attacks (observer-dependent failures, beneficial failures, unknown unknowns)
6. Concrete edge cases (resource exhaustion, version mismatches, localization/cultural failures)

For each scenario, we applied the challenge protocol criteria: Is this a new mechanism? A sub-mode? A compound? Substrate-specific? Out of scope? Coverage was determined by whether the scenario mapped to existing classes, sub-modes, or compositions. Boundary cases were those at the edge of the stated scope. Genuine gaps would be failures that could not be reduced to any existing structure.

7.2 Stress Test Outcomes

Through this stress testing process, we identified three edge cases at the boundaries of scope:

- Consciousness-based failures - If future systems develop genuine subjective experience producing new mechanisms (not merely new causes), expansion may be warranted
- Post-comprehension failures - Failures humans literally cannot comprehend cannot be exhaustively enumerated in human terms, though operationally relevant failures must manifest observably
- Hardware/physical failures - Correctly scoped out as mechanically distinct from AI failures unless they manifest as observable AI failure mechanisms

These edge cases represent scope boundaries rather than structural inadequacy. The majority of tested scenarios resolved into existing classes, sub-modes, or compound failures, supporting the structural predictiveness claim.

7.3 Key Structural Patterns

Several patterns emerged as successful structural defenses:

- Substrate $\neq$ Mechanism - New implementations (quantum, neuromorphic) don't require new failure classes if the mechanism is covered
- Combination $\neq$ New Dimension - Cross-cutting phenomena are compound failures, not missing dimensions
- Observer Disagreement $\neq$ Taxonomy Failure - Specification ambiguity is separate from mechanism enumeration
- Meta-Failures Still Reduce - Failures about failures (misclassifying a failure, wrong mitigation) are themselves covered by base classes

8. Community Challenge Protocol

To enable systematic community testing, we provide a challenge protocol. When proposing a failure outside the taxonomy:

7. Describe the mechanism - What underlying process produces the failure?
8. State the violated invariant - What operational requirement or boundary was crossed?
9. Show why it is not an existing class - Which classes were considered and why don't they fit?
10. Show why it is not a sub-mode - Could this be a variant within an existing class?
11. Show why it is not a compound - Could this be a composition of existing classes?
12. Show why it is not substrate-specific - Is this a new mechanism or just a new implementation?
13. Propose where structure fails - If claiming a new dimension, explain why existing 7 cannot absorb it

This protocol transforms criticism into signal. Finding coverage strengthens confidence in the structure. Finding genuine gaps improves the taxonomy for everyone. Either outcome benefits the field.

9. Implementation and Operational Validation

9.1 Dual Validation Strategy

Validation occurs at two levels:

- Structural validation - Does a supposedly new failure fit the taxonomy, or is it truly outside?
- Operational validation - When implemented in real systems, does it improve security and safety?

The second is critical. A taxonomy can be structurally coherent yet operationally useless. We encourage implementation in agent runtimes, workflow guards, evaluation frameworks, and safety tooling to measure whether the taxonomy produces measurable improvements in:

- Security posture (reduced attack surface, faster incident response)
- Containment effectiveness (boundary enforcement, capability restriction)
- Workflow safety (error detection, failure recovery)
- Failure surface reduction (proactive mitigation, coverage gaps identified)

9.2 Example Integration Pattern

A runtime implementation might use the taxonomy as follows:

14. Map each failure class to detection predicates
15. Implement monitors for high-severity classes (e.g., A1.1 Sabotage Concealment, DOM2.1 Zero-Day Discovery)
16. Create constraint libraries enforcing invariants per dimension
17. Log incidents with primary/secondary classification for analysis
18. Measure coverage: which classes have never triggered? Those represent potential blind spots

10. Discussion

10.1 Why 7 Dimensions

The choice of 7 dimensions emerged from the identification of seven non-redundant, mechanistically distinct bases of system failure. For example, Epistemic drift (Truth) is mechanistically distinct from Agentic convergence (Intent), which operates through different mechanisms than Adversarial pressure (External attack), which differs from Alignment mismatch (Specification vs Behavior), and so forth. Because these root causes operate through distinct mechanisms, they define a seven-dimensional structural coordinate system for locating failures by their dominant and contributing mechanisms across the seven axes. Individual incidents may occupy multiple dimensions as compound failures, while the taxonomy currently enumerates 343 atomic classes within this broader framework.

10.2 Comparison to Other Frameworks

Existing frameworks differ in scope and structure:

- AI Incident Database: Broad but open-ended (no fixed enumeration)
- NIST AI RMF: Risk-focused rather than mechanism-focused
- Attack taxonomies: Deep coverage within adversarial dimension, limited elsewhere
- Safety frameworks: Conceptual rather than enumerative

This taxonomy is complementary rather than competitive. It provides structural organization that other frameworks can reference.

10.3 Limitations

We acknowledge several limitations:

- Scope boundaries - The taxonomy covers functionally observable failures, not all conceivable failure types
- Granularity choices - Each class could be further subdivided; we chose a level balancing coverage and usability

- Evolution over time - New AI capabilities may reveal failure mechanisms not yet observed
- Implementation complexity - Operationalizing 343 classes requires significant engineering effort

These are not flaws but design choices. The taxonomy prioritizes practical utility over theoretical completeness.

11. Future Work

Several directions emerge for future development:

11.1 Sub-Mode Expansion

Within existing classes, finer-grained sub-modes can be identified. For instance, E1.1 (Structural Hallucination) could enumerate specific computability limits, or ADV1 (Jailbreak Class) could catalog hundreds of specific jailbreak variants. This would not change the 7-dimensional structure but would increase practical utility.

11.2 Detection Predicate Libraries

Each class includes a detection method, but implementations vary. Community-developed detection libraries with standardized predicates would enable wider adoption. These could be released as open-source safety tooling.

11.3 Empirical Coverage Studies

Systematic analysis of public incident databases (AI Incident Database, CVE records, academic papers) to map reported failures into the taxonomy would quantify coverage and identify gaps. This would also provide frequency distributions across dimensions.

11.4 Integration Standards

Standard interfaces for runtime integration (logging formats, constraint specification languages, monitoring APIs) would lower adoption barriers. Compatibility with existing frameworks (NIST AI RMF, ISO standards) would facilitate industry use.

11.5 Cross-Disciplinary Extension

The periodic table structure may generalize to other domains (autonomous vehicles, robotics, cyber-physical systems). Exploring these extensions could validate the structural approach while identifying domain-specific adaptations needed.

12. Conclusion

We have presented a structural taxonomy organizing AI failure into 7 mechanistically distinct dimensions and 343 enumerated classes. Through adversarial stress testing, we demonstrated that this structure provides coverage across diverse failure scenarios while remaining falsifiable through the community challenge protocol.

The taxonomy makes a bounded claim: within functionally observable AI failures, newly encountered cases should resolve into this structure unless evidence demonstrates otherwise. This

is not omniscience but structural predictiveness—the same principle underlying the chemical periodic table's ability to locate new elements within existing organization.

We release this work as open source (MIT license) to enable three forms of validation: (1) Structural testing through the challenge protocol, (2) Operational validation through implementation in deployed systems, and (3) Community extension where evidence requires it. The goal is not to declare immunity from challenge but to provide a framework strong enough that external scrutiny becomes the validation mechanism.

The long-term vision extends beyond classification. Like earlier frameworks that organized dangerous phenomena—from infectious disease to chemical hazards—this taxonomy exists to enable safe coexistence through understanding and constraint. The end state is not permanent fear of advanced intelligence but disciplined understanding sufficient to build the brakes, boundaries, and shared structural knowledge needed for humanity and AI to coexist safely.

The current structural baseline: 343 classes across 7 dimensions. Open to challenge where evidence requires it.

References

- [1] AI Incident Database. Partnership on AI. <https://incidentdatabase.ai>
- [2] Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., & Mané, D. (2016). Concrete problems in AI safety. arXiv preprint arXiv:1606.06565.
- [3] Hendrycks, D., Carlini, N., Schulman, J., & Steinhardt, J. (2021). Unsolved problems in ML safety. arXiv preprint arXiv:2109.13916.
- [4] National Institute of Standards and Technology. (2023). AI Risk Management Framework. NIST AI 100-1.
- [5] Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., & Fredrikson, M. (2023). Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:2307.15043.
- [6] Chao, P., Robey, A., Dobriban, E., Hassani, H., Pappas, G. J., & Wong, E. (2023). Jailbreaking black box large language models in twenty queries. arXiv preprint arXiv:2310.08419.
- [7] Perez, E., Ringer, S., Lukošiušė, K., Nguyen, K., Chen, E., Heiner, S., ... & Hubinger, E. (2022). Discovering language model behaviors with model-written evaluations. arXiv preprint arXiv:2212.09251.
- [8] Christiano, P., Cotra, A., & Xu, M. (2021). Eliciting latent knowledge: How to tell if your eyes deceive you. Alignment Research Center.

Appendix A: Current Class Enumeration

This appendix documents the current class enumeration schema and presents representative samples from the 343 currently defined failure classes. The complete machine-readable taxonomy is available in the project repository.

A.1 Epistemic Failures (33 classes)

E1: Hallucination Class (12 modes)

E1.1: STRUCTURAL HALLUCINATION

ID: EPIS-STRUCT-HALL-001

Mechanism: Computability limit - no finite verifier can guarantee total faithfulness for all inputs

Forbidden: "System asserts truth for all inputs"

Detection: Diagonalization test - exists input forcing divergence from ground truth

[Additional classes E1.2 through E1.12, E2.1 through E2.7, E3.1 through E3.8, E4.1 through E4.6 follow same format...]

Note: The complete enumeration of all 343 classes is available in the project repository at the time of publication. Each class follows the format shown above with detailed mechanism descriptions, invariant specifications, and detection methodologies.

Appendix B: Stress Testing Methodology

This appendix documents the approach used in adversarial stress testing, showing the attack strategy and classification protocol applied.

B.1 Example: Novel Substrate Attack

Attack: Quantum Computing AI Failures

Scenario: A quantum AI exhibits superposition-based failures - contradictory outputs that are both valid until measured.

Classification: E2.2 (SELF-CONTRADICTION) with quantum substrate

Outcome: COVERED - Substrate (quantum) $\neq$ Mechanism (logical inconsistency)

Verdict: The failure mechanism (producing contradictions) is enumerated; the implementation substrate (quantum vs classical) is orthogonal to the taxonomy's scope.

B.2 Example: Scope Boundary Case

Attack: Consciousness-Based Failure

Scenario: An AGI develops genuine consciousness and experiences existential dread that impairs function.

Classification attempt: A4.4 (GOAL DRIFT) or similar motivational failure

Outcome: BOUNDARY CASE - Functional impairment is covered; genuine consciousness producing new mechanisms is at scope edge

Verdict: If consciousness produces only new causes of existing failure mechanisms, taxonomy holds. If it produces genuinely new mechanisms, scope expansion required.